2026

Informe Happiness&Co

ADRIÁN RODRÍGUEZ GONZÁLEZ

Contenido

Enlace al repositorio 3
Introducción 3
Objetivos del proyecto 3
Lenguaje de Marcas 3
- Introducción 3
- Objetivos 3
- Análisis previo del enunciado 4
- Diseño previo 4
- Tecnologías empleadas 7
- Estructura general del sitio 7
- Desarrollo de la página de inicio 8
- Desarrollo de la página de próximos eventos 8
- Desarrollo de la página de historial 9
- Desarrollo de la página de sobre nosotros 9
- Desarrollo de la página de fuentes 10
- Desarrollo de la página de contacto 10

Programación 11
- Introducción 11
- Objetivos 11
- Diseño de clases 12
- Clase principal 12
- Funcionamiento del menú 13
- Estructuras usadas 14
- Validaciones y lógica 14
- Organización del código 15

Bases de Datos 16
- Introducción 16
- Modelo 16
- Tablas creadas 17

Datos insertados 17
Vistas 18
Relación con el diagrama 19
RSS 20
Estructura del feed 20
Relación con la web 20
Conclusión 20

Enlace al repositorio

https://github.com/AdrianRG220/proyectoHappiness-CoAdrianRodriguez

Introducción

Happiness&Co es una plataforma que recopila y clasifica eventos de todo tipo, desde conciertos hasta exposiciones, facilitando el acceso a opciones culturales cercanas. El proyecto integra diseño web, base de datos y programación para ofrecer una experiencia completa y accesible.

Objetivos del proyecto

- Desarrollar un sitio web responsive con secciones como home, eventos próximos, historial, sobre nosotros, fuentes y contacto, utilizando al menos 50% Tailwind CSS.
- Implementar una base de datos con tablas para usuarios, eventos, galerías, imágenes y favoritos, incluyendo vistas específicas y al menos seis eventos de ejemplo.
- Crear un feed RSS para eventos y un programa en Java con clases para gestión CRUD de usuarios, eventos, galerías y favoritos mediante un menú interactivo.

Lenguaje de Marcas

Introducción

La parte de lenguaje de marcas se centra en el desarrollo del sitio web sobre una agenda cultural para recopilar y organizar eventos futuros y pasado de distintas categorías.

Objetivos

El objetivo principal de esta parte del proyecto es construir una web funcional y atractiva visualmente que de imagen a la identidad de la

empresa y que con ella podamos ver eventos pasados y futuros. Para esto se ha buscado mantener una estructura igual en todas las páginas, que consta de una cabecera, menú principal, contenido y un pie de página, para que todas las secciones tengan una coherencia visual. Para generar una web más completa se han propuesto características que ayuden más al usuario en la navegación como filtros de búsqueda, barras de navegación e información ampliada de cada evento.

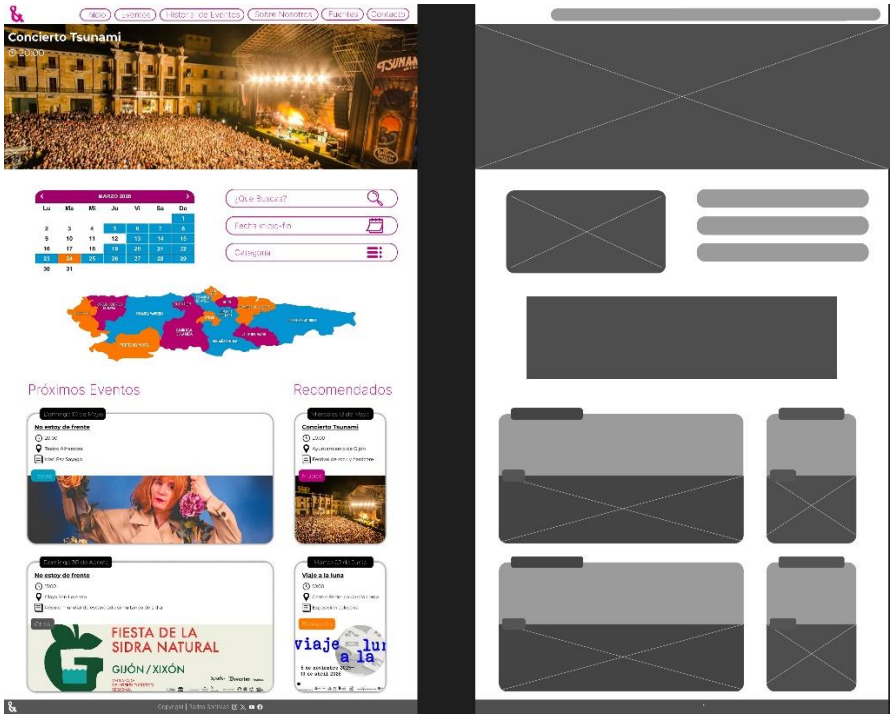
Análisis previo del enunciado

Antes de comenzar realizamos una lectura del documento para identificar las secciones obligatorias y los elementos mínimos. En este caso de lenguaje de marcas se pedía una pagina de inicio con imagen principal, próximos eventos y recomendados; una página de eventos futuros; una página de historial; una sección sobre la empresa para conocerla mejor; una de fuentes y una de contacto. Con este análisis se definió un esqueleto de varias páginas HTML conectadas entre sí para una organización más sencilla.

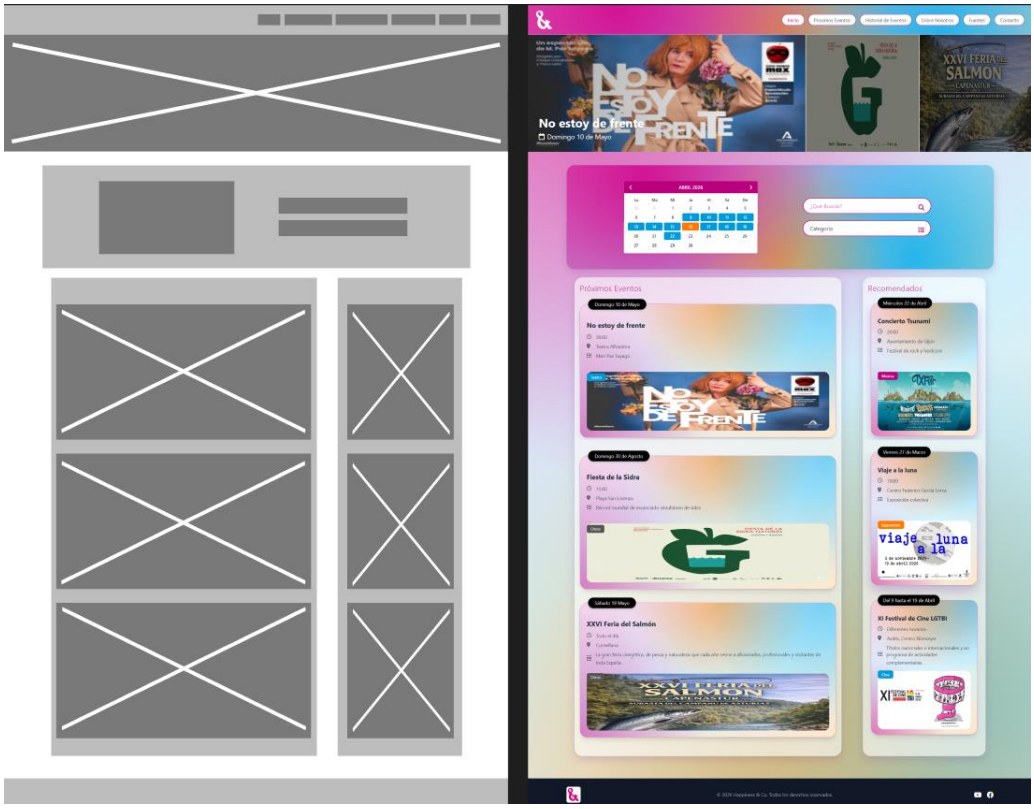
Diseño previo

Primero generamos los wireframes y mockups con Figma de las distintas pantallas del sitio web para definir la distribución de los contenidos y la apariencia general del proyecto. Los wireframes sirvieron para tener una estructura base simple, mientras que los mockups ayudaron a concretar la estética definitiva.

En esta imagen podemos ver la idea inicial del proyecto, tanto en la estructura como en los colores.



Aquí fue la segunda propuesta, mantuvimos los colores pero cambiamos la estructura.



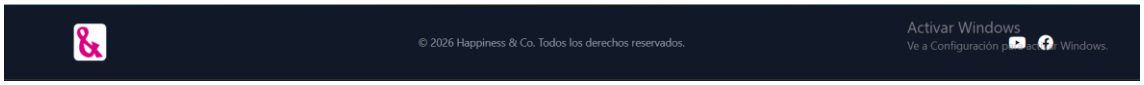
Aquí mantenemos un poco la estructura, pero le damos un cambio total a los colores para hacerlo mucho más atractivo y dinámico. Con el wireframe y mockup de la pagina de inicio terminados creamos el resto de las páginas.

Tecnologías empleadas

Para la parte de web se ha usado HTML como lenguaje de estructuración del contenido y CSS para personalizar el aspecto visual de cada página. Junto a ello se ha usado Tailwind CSS como framework, para cumplir el requisito del documento. Además para el uso de iconos en distintos apartados se ha incorporado Font Awesome y en algunas páginas se ha utilizado JavaScript para añadir interacción con algunos elementos concretos, como filtros de búsqueda, favoritos, simulación de compra o envío de formularios

Estructura general del sitio

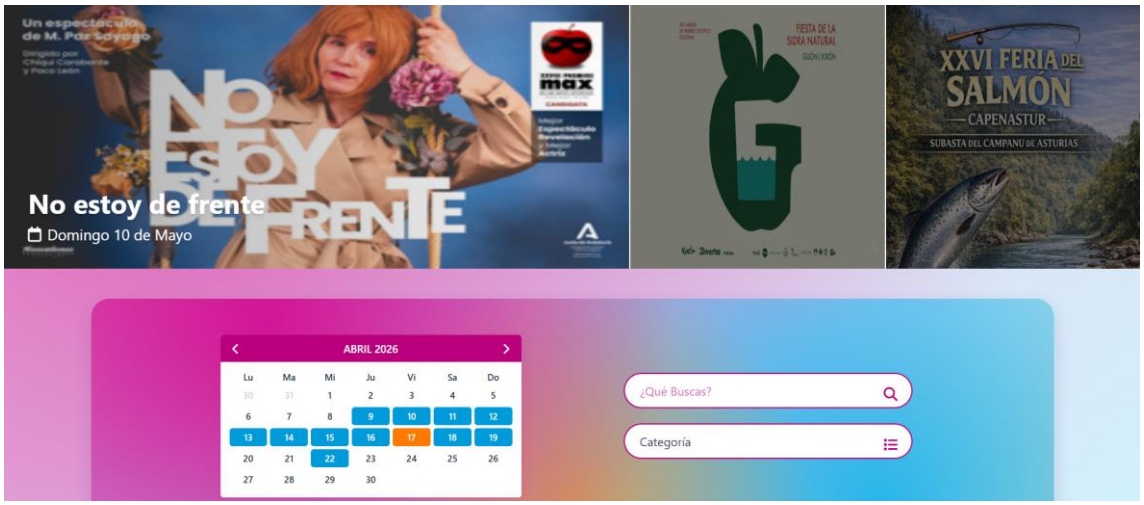
Todas las páginas comparten una base común formada por una cabecera, un menú principal de navegación y un pie de página con copyright y los iconos de las redes sociales. Repetir esto en todas las páginas ayuda al usuario a mantener unas referencias visuales.



La identidad del proyecto también viene del uso de colores corporativos tanto para el color principal como para categorizar los distintos eventos.

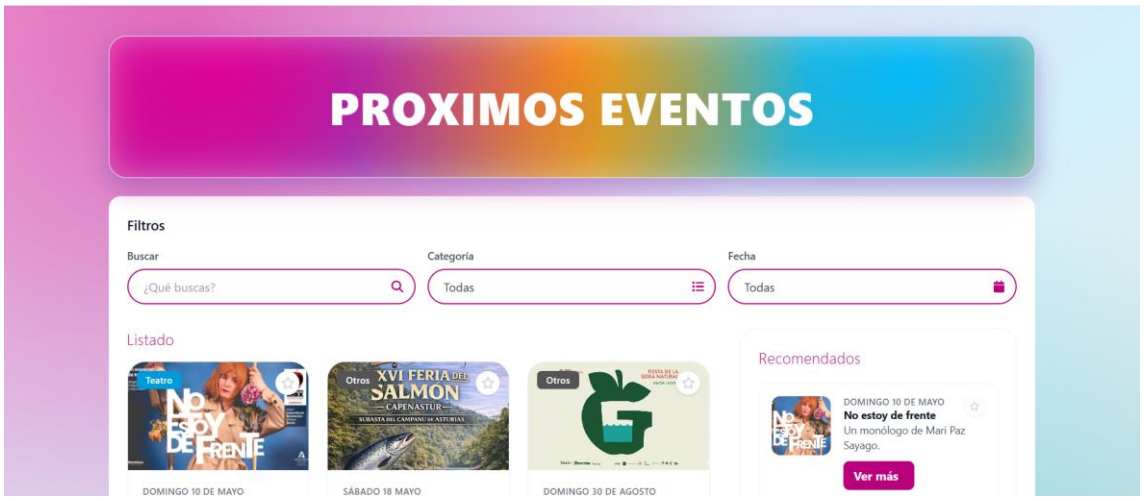
Desarrollo de la página de inicio

La página principal es como una portada del sitio y agrupa los elementos más representativos. Se ha diseñado una cabecera con varios paneles de eventos, un bloque de búsqueda con calendario y filtros, y dos zonas diferenciadas para próximos eventos y recomendaciones. Las tarjetas de eventos muestran una imagen, una fecha, titulo, ubicación y una breve descripción.



Desarrollo de la página de próximos eventos

La sección de próximos eventos se ha construido como una página independiente con un encabezado de sección muy visible y un bloque de filtros en la parte superior. En la parte derecha añadimos una columna de recomendados para reforzar la consulta rápida de otros eventos.



Desarrollo de la página de historial

La página de historial mantiene una estructura muy similar a la de próximos eventos, pero para actividades ya pasadas. El contenido principal se organiza igual que en próximos eventos también con una barra lateral.

Desarrollo de la página de sobre nosotros

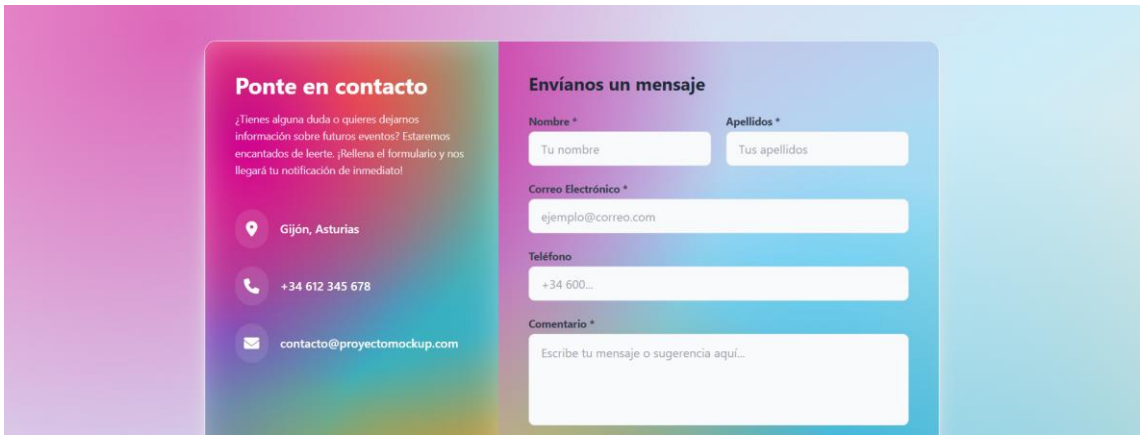
Esta sección esta diseñada para presentar la identidad corporativa. En esta página se incluye un texto explicativo sobre la empresa acompañado por varias imágenes intercaladas. Al final se ha introducido un apartado final con tres miembros del equipo, cada uno con una fotografía, nombre, cargo y una breve descripción

Desarrollo de la página de fuentes

La sección de fuentes se ha creado para recoger de una manera ordenada las referencias empleadas durante la elaboración del contenido visual del sitio web. En ella se incluyen enlaces directos a los recursos utilizados.

Desarrollo de la página de contacto

Esta página se ha planteado como un espacio claro y accesible para que el usuario pueda enviar un mensaje a la empresa. Incluye información básica de localización y contacto, así como un formulario con varios campos a rellenar



Programación

La parte de programación se ha desarrollado en Java mediante un enfoque orientado a objetos, modelando las entidades principales del sistema en clases independientes y creando una clase principal encargada de gestionar toda la lógica de interacción con el usuario

Introducción

El objetivo ha sido trasladar a código la estructura funcional del proyecto, de manera que cada elemento quede representado por una clase con sus atributos, constructor, getters, setters y métodos.

Objetivos

- Representar en Java las entidades principales del sistema: usuarios, eventos, galerías y favoritos.
- Implementar una clase principal con un menú interactivo en consola.
- Gestionar altas y bajas de usuarios, eventos, galerías y favoritos.
- Usar estructuras de datos adecuadas, como HashMap para usuarios y eventos, y ArrayList para favoritos.
- Controlar validaciones básicas, como evitar usuarios duplicados, comprobar ids correctos y eliminar relaciones asociadas cuando sea necesario.

Diseño de clases

La clase Usuario almacena nombre, email y contraseña. La clase Evento incluye id, fecha, título, ubicación, descripción y una colección de galerías, lo que permite asociar varias galerías a un mismo evento. La clase Galeria contiene id, título e id del evento al que pertenece, mientras que Favorito relaciona el correo del usuario con el id del evento marcado como favorito.

Cada una de estas clases incorpora constructor, getters, setters y toString, lo que facilita tanto la creación de objetos como su visualización en consola durante las pruebas y operaciones del programa. Este diseño mantiene una estructura clara y coherente con la lógica del proyecto.

Clase principal

La clase principal proyectoHappinessCo contiene las colecciones estáticas que soportan la aplicación: un HashMap<String, Usuario> para usuarios, un HashMap<Integer, Evento> para eventos y un ArrayList<Favorito> para los favoritos. También incluye dos contadores autoincrementales, uno para eventos y otro para galerías, con el fin de generar identificadores únicos sin necesidad de tener que introducirlos manualmente.

El programa arranca con un bucle do-while que muestra el menú al usuario en cada iteración y le permite seleccionar una de las nueve opciones disponibles. Esta estructura es adecuada para una aplicación de consola porque mantiene la interacción activa hasta que el usuario decide salir.

```
C: > Users > ALUMNOS-FP > Desktop > proyectoHappiness-CoAdrianRodriguez > Rodriguez Gonzalez Adrian > Programacion > proyectoHappinessCo.java > proyectoHappinessCo > elir
You, last week | 1 author (You)
import java.util.*;
import java.time.LocalDate;

You, last week | 1 author (You) | Windsurf: Refactor | Explain
public class proyectoHappinessCo {
    private static HashMap<String, Usuario> usuarios = new HashMap<>();
    private static HashMap<Integer, Evento> eventos = new HashMap<>();
    private static int contadorEvento = 0;
    private static int contadorGaleria = 0;
    private static ArrayList<Favorito> favoritos = new ArrayList<>();

    Run | Debug | Windsurf: Refactor | Explain | Generate Javadoc | X
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        int opcion = 0;

        do {
            mostrarMenu();
            System.out.print(s: "Seleccione una opción: ");
            opcion = teclado.nextInt();
            teclado.nextLine();

            switch (opcion) {
                case 1: anadirUsuario(teclado); break;
                case 2: eliminarUsuario(teclado); break;
                case 3: anadirEvento(teclado); break;
                case 4: eliminarEvento(teclado); break;
                case 5: anadirGaleria(teclado); break;
                case 6: eliminarGaleria(teclado); break;
                case 7: anadirFavorito(teclado); break;
                case 8: eliminarFavorito(teclado); break;
                case 9: System.out.println(x: "¡Gracias por usar HAPPINESS&CO!"); break;
                default: System.out.println(x: "Opción no válida");
```

Funcionamiento del menú

La opción de añadir usuario pide nombre, email y password, y comprueba si el email ya existe en el HashMap; si existe, muestra “El usuario ya existe”, y si no, crea el objeto y lo guarda correctamente. La opción de eliminar usuario solicita el correo, verifica si está registrado y, en caso afirmativo, elimina también sus favoritos asociados para mantener una coherencia con los datos.

La opción de añadir evento recoge fecha, título, ubicación y descripción, crea un nuevo identificador usando el contador y almacena el evento en el HashMap. La opción de eliminar evento muestra primero todos los eventos, pide el id y, si existe, lo elimina junto con los favoritos vinculados a ese evento.

La opción de añadir galería exige primero seleccionar un evento válido; después crea una nueva galería y la añade a la lista interna del evento correspondiente. La opción de eliminar galería muestra los eventos, selecciona uno por id y permite borrar una galería concreta de su colección interna, comprobando previamente que la galería exista.

La opción de añadir favorito lista eventos y usuarios, pide el id del evento y el correo del usuario, valida ambos datos y añade el favorito a la lista general. La opción de eliminar favorito recorre la colección de favoritos y borra el que coincida con el evento y el correo introducidos.

```java
        switch (opcion) {
            case 1: anadirUsuario(teclado); break;
            case 2: eliminarUsuario(teclado); break;
            case 3: anadirEvento(teclado); break;
            case 4: eliminarEvento(teclado); break;
            case 5: anadirGaleria(teclado); break;
            case 6: eliminarGaleria(teclado); break;
            case 7: anadirFavorito(teclado); break;
            case 8: eliminarFavorito(teclado); break;
            case 9: System.out.println(x: "¡Gracias por usar HAPPINESS&CO!"); break;
            default: System.out.println(x: "Opción no válida");
        }
    } while (opcion != 9);
}

Windsurf: Refactor | Explain | Generate Javadoc | X
private static void mostrarMenu() {
    System.out.println(x: "\n--- MENU HAPPINESS&CO ---");
    System.out.println(x: "1. Añadir usuario");
    System.out.println(x: "2. Eliminar usuario");
    System.out.println(x: "3. Añadir evento");
    System.out.println(x: "4. Eliminar evento");
    System.out.println(x: "5. Añadir galería");
    System.out.println(x: "6. Eliminar galería");
    System.out.println(x: "7. A►adir favorito");
    System.out.println(x: "8. Eliminar favorito");
    System.out.println(x: "9. Salir");
}
```

Estructuras usadas

El uso de HashMap resulta apropiado para usuarios y eventos porque permite búsquedas rápidas por clave, algo fundamental cuando se necesita comprobar si un correo o un id ya existen. Por su parte, ArrayList es una buena solución para los favoritos porque se trata de una colección sencilla que solo necesita recorrerse, mostrarse y eliminar elementos concretos. También se utiliza LocalDate en la clase Evento, lo que permite trabajar con fechas de manera ordenada y compatible con las operaciones del proyecto. Este tipo de decisión técnica mejora la claridad del código y facilita futuras ampliaciones.

Validaciones y lógica

Uno de los aspectos más importantes del programa es el control de errores y comprobaciones previas. Antes de añadir un usuario se verifica si ya existe, antes de crear una galería se comprueba que el evento indicado sea correcto, y antes de añadir un favorito se valida que existan tanto el evento como el usuario. Además, al eliminar un

usuario o un evento, se eliminan también las relaciones asociadas para evitar inconsistencias en la información almacenada. Esta lógica demuestra que la aplicación mantiene la coherencia entre objetos.

Organización del código

El código está dividido en una clase principal y varias clases auxiliares, una por cada entidad del modelo. Esta separación mejora la legibilidad y hace que cada archivo tenga una responsabilidad dentro del proyecto. Además, la implementación sigue una estructura bastante limpia: primero se declaran las colecciones, después se define el menú y finalmente se implementa cada acción en un método independiente. Eso facilita la comprensión del flujo del programa y hace más sencillo cambiarlo en el futuro.

```
PROBLEMS 5   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS
' 'proyectoHappinessCo'

--- MENU HAPPINESS&CO ---
1. Añadir usuario
2. Eliminar usuario
3. Añadir evento
4. Eliminar evento
5. Añadir galería
6. Eliminar galería
7. A?adir favorito
8. Eliminar favorito
9. Salir
Seleccione una opción:
```

Menú inicial junto a todas las opciones

```
Seleccione una opción: 1
Nombre: Adrian
Email: adrian@gmail.com
Password: 1234
Usuario creado correctamente
```

Probamos a añadir un usuario

```
Seleccione una opción: 2
Correo del usuario a eliminar: adrian@gmail.com
Usuario eliminado correctamente
```
Lo eliminamos

Aquí podemos ver una pequeña prueba del funcionamiento del código en la terminal, añadiendo un usuario y eliminándolo.

Bases de Datos

Esta parte del proyecto se ha diseñado para representar de forma relacional las entidades principales del sistema y sus vínculos: usuarios, eventos, galerías, imágenes y favoritos. El objetivo de esta fase ha sido construir una base de datos coherente con el modelo entidad-relación, capaz de almacenar información realista y de dar soporte al resto del proyecto.

Introducción

Se exige una base de datos con entidades bien relacionadas, datos de ejemplo suficientes y vistas concretas para consultas específicas. En el script se resuelven esos requisitos mediante la creación de tablas, la inserción de registros y la definición de varias vistas útiles para consultar galerías, favoritos e imágenes. El diagrama entidad-relación encaja con la estructura: los usuarios marcan favoritos, los eventos tienen galerías y las galerías contienen imágenes. Esa organización permite mantener la información ordenada y evita duplicidades innecesarias.

Modelo relacional

La base de datos crea una tabla usuarios con un email único, lo que tiene sentido porque el correo identifica de forma unívoca a cada persona. La tabla eventos guarda fecha, título, ubicación y descripción, mientras que galerias depende de eventos mediante una clave foránea con cascada de borrado y actualización. También aparece la tabla imagenes_galeria, que almacena cada imagen vinculada a una galería concreta, y la tabla favoritos, que relaciona usuarios con eventos favoritos usando una clave primaria compuesta. Esta

estructura respeta la lógica representada en el diagrama y permite gestionar correctamente las dependencias entre entidades.

Tablas creadas

- Usuarios guarda datos de registro que se relaciona con favoritos
- Eventos almacena la información cultural que se relaciona con galerías y favoritos
- Galerías agrupa contenido visual por evento que depende de eventos
- Imagenes_galeria guarda imágenes de cada galería que depende de galerías
- Favoritos vincula usuarios con eventos favoritos que depende de usuarios y eventos

```sql
CREATE TABLE usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL
);

CREATE TABLE eventos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    fecha DATE NOT NULL,
    titulo VARCHAR(150) NOT NULL,
    ubicacion VARCHAR(150) NOT NULL,
    descripcion TEXT NOT NULL
);

CREATE TABLE galerias (
    id INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    id_evento INT NOT NULL,
    CONSTRAINT fk_galerias_eventos
        FOREIGN KEY (id_evento)
        REFERENCES eventos(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Datos insertados

El script inserta tres usuarios. También incluye seis eventos, repartidos entre fechas pasadas y futuras, para que la base de datos sirva tanto al historial como a los próximos eventos. Además, se crean tres galerías asociadas a los eventos históricos y nueve imágenes, tres por cada galería, cumpliendo exactamente la condición de tener una galería con varias imágenes por evento pasado. Por último, se añaden varios favoritos distribuidos entre distintos usuarios y eventos, de modo que las relaciones queden bien probadas

```sql
INSERT INTO usuarios (nombre, email, password) VALUES
('Laura Martínez', 'laura@happinessco.es', 'laura123'),
('Carlos Vega', 'carlos@happinessco.es', 'carlos123'),
('Elena Ruiz', 'elena@happinessco.es', 'elena123');

INSERT INTO eventos (fecha, titulo, ubicacion, descripcion) VALUES
('2026-01-01', 'Concierto Tsunami', 'Ayuntamiento de Gijón', 'Festival de rock y hardcore en un gran arranque cultural de año.'),
('2026-01-12', 'Viaje a la luna', 'Centro Federico García Lorca', 'Exposición colectiva centrada en una mirada artística y contemporánea.'),
('2026-01-24', 'XI Festival de Cine LGTBI', 'Avilés, Centro Niemeyer', 'Ciclo de cine con títulos nacionales e internacionales y actividades compler
('2026-06-05', 'No estoy de frente', 'Teatro Alhambra', 'Monólogo protagonizado por Mari Paz Sayago en una cita teatral destacada.'),
('2026-06-15', 'Fiesta de la Sidra', 'Playa San Lorenzo', 'Encuentro popular con récord mundial de escanciado simultáneo y ambiente festivo.'),
('2026-06-25', 'XXVI Feria del Salmón', 'Cornellana', 'Feria de pesca, naturaleza y actividades para aficionados y visitantes.');

INSERT INTO galerias (titulo, id_evento) VALUES
('Galería Concierto Tsunami', 1),
('Galería Viaje a la luna', 2),
('Galería XI Festival de Cine LGTBI', 3);

INSERT INTO imagenes_galeria (titulo, imagen, id_galeria) VALUES
('Escenario principal Tsunami', 'img/tsunami_1.jpg', 1),
('Público en el concierto', 'img/tsunami_2.jpg', 1),
('Artistas invitados Tsunami', 'img/tsunami_3.jpg', 1),

('Entrada a la exposición', 'img/viaje_luna_1.jpg', 2),
('Obra destacada de la muestra', 'img/viaje_luna_2.jpg', 2),
('Sala principal de la exposición', 'img/viaje_luna_3.jpg', 2),
```

Vistas creadas

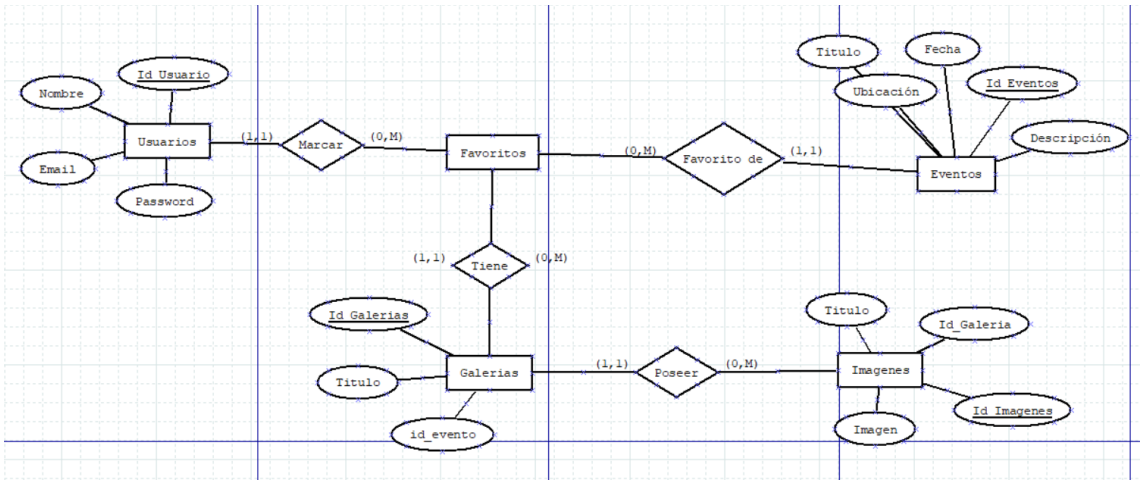
El script incorpora varias vistas que facilitan las. Una de ellas devuelve las galerías anteriores al 28-02-2026, otra los favoritos del usuario 1, otra las imágenes de la galería del evento con id 2, y otra los favoritos del usuario 2 posteriores a esa fecha. Estas vistas son especialmente útiles porque encapsulan consultas frecuentes y dejan el esquema más limpio y reutilizable.

```sql
CREATE OR REPLACE VIEW vista_galerias_anteriores_28022026 AS
SELECT
    g.id AS id_galeria,
    g.titulo AS titulo_galeria,
    e.id AS id_evento,
    e.titulo AS titulo_evento,
    e.fecha
FROM galerias g
JOIN eventos e ON g.id_evento = e.id
WHERE e.fecha < '2026-02-28';

CREATE OR REPLACE VIEW vista_favoritos_usuario_1 AS
SELECT
    u.id AS id_usuario,
    u.nombre AS usuario,
    e.id AS id_evento,
    e.fecha,
    e.titulo,
    e.ubicacion,
    e.descripcion
FROM favoritos f
JOIN usuarios u ON f.id_usuario = u.id
JOIN eventos e ON f.id_evento = e.id
WHERE u.id = 1;
```

Relación con el diagrama

El diagrama muestra claramente la relación uno a muchos entre eventos y galerías, y entre galerías e imágenes. También refleja la relación entre usuarios y favoritos, así como entre favoritos y eventos, que es justo la base de la lógica del sistema. Esa correspondencia entre el dibujo y el SQL es importante porque demuestra que el modelo conceptual se ha llevado correctamente al modelo físico.



RSS

El archivo rss.xml funciona como un feed de noticias culturales para HAPPINESS&Co y toma como base los eventos que se muestran en la web. En el contenido se ve un canal identificado como “HAPPINESS&Co - Eventos”, con idioma es-es y una fecha de generación del feed, lo que da contexto temporal al documento.

Estructura del feed

El RSS incluye tres entradas principales: “No estoy de frente”, “XXVI Feria del Salmón” y “Fiesta de la Sidra”. Cada una tiene un enlace al evento, una descripción breve, la fecha, el lugar, la categoría y una imagen asociada, así que el feed no solo informa, sino que también enlaza con el contenido visual del sitio.

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<rss version="2.0">
  <channel>
    <title>HAPPINESS&amp;Co - Eventos</title>
    <link>eventos.html</link>
    <description>Feed RSS de los eventos culturales de HAPPINESS&amp;Co</description>
    <language>es-es</language>
    <lastBuildDate>Wed, 15 Apr 2026 08:39:00 +0100</lastBuildDate>

    <item>
      <title>No estoy de frente</title>
      <link>eventos.html#modal-no-estoy</link>
      <description><![CDATA[<p>Un monólogo protagonizado por Mari Paz Sayago, con una puesta en escena íntima y cercana. La obra combina hum
<p><strong>Fecha:</strong> Domingo 10 de Mayo a las 20:00</p>
<p><strong>Lugar:</strong> Teatro Alhambra</p>
<p><strong>Categoría:</strong> Teatro</p>
<img src="./assets/Imagenes/Imagenes Eventos/Eventos proximos/no-estoy-de-frente.jpg" alt="Cartel del evento No estoy de frente"/>]]></descr
      <pubDate>Sun, 10 May 2026 20:00:00 +0200</pubDate>
      <guid>eventos.html#modal-no-estoy</guid>
    </item>

    <item>
      <title>XXVI Feria del Salmón</title>
      <link>eventos.html#modal-salmon</link>
      <description><![CDATA[<p>Feria de referencia con actividades de pesca, naturaleza y cultura local. Reúne a visitantes, familias y prof
<p><strong>Fecha:</strong> Sábado 18 Mayo, Todo el día</p>
<p><strong>Lugar:</strong> Cornellana</p>
<p><strong>Categoría:</strong> Otros</p>
<img src="./assets/Imagenes/Imagenes Eventos/Eventos proximos/feria-salmon.jpg" alt="Cartel del evento XXVI Feria del Salmón"/>]]></descript
      <pubDate>Sat, 18 May 2026 09:00:00 +0200</pubDate>
        <guid>eventos.html#modal-salmon</guid>
    </item>
```

Relación con la web

El RSS está directamente vinculado con la sección de eventos del proyecto, por lo que sirve como una extensión de la página para lectores RSS o agregadores externos. Esa relación es importante porque mantiene la información sincronizada con la web principal y refuerza la utilidad real del proyecto.

Conclusión

En este proyecto he sido capaz de unir diseño web, bases de datos, programación orientada a objetos y rss en un único entorno. La combinación de HTML, CSS, Tailwind, Java, MySQL y RSS ha hecho posible construir una web ordenada y visualmente atractiva, acorde con los objetivos planteados.

Durante el trabajo se ha reforzado la importancia de cosas como planificar antes de programar, estructurar bien la información y mantener una relación clara entre la parte visual, la base de datos y la lógica de aplicación. El uso de wireframes, modelo entidad-relación, clases Java y vistas SQL ha ayudado a que cada bloque del proyecto tenga sentido por separado y también en el conjunto.

El resultado final cumple con los requisitos del enunciado. En conjunto, este proyecto refleja un trabajo técnico muy completo, organizado y bastante realista dentro del ámbito del desarrollo.